Institute of Field Robotics, King Mongkut's University of Technology Thonburi
126 Pracha Uthit Rd, Bang Mot, Thung Khru, Bangkok, Thailand 10140
Senior Thesis FRAB (FIBO Robotics and Automation: Bachelor)

A study of Torque-Control system on UR5 for the drawing task

Jirattanun Leeudomwong, 66340500009
Bharut Aursudkit, 66340500040
Nuttapruet Puttiwarodom, 66340500018

Abstract
Industrial robot manipulators have countless applications nowadays. One of the basic tasks to test the accuracy of a robot manipulator is to perform a drawing task, which can be done with only position control alone. This project focuses on using a UR5 industrial robot manipulator to perform the drawing task. The challenge this project represents is controlling the force between the pen (end-effector) and the canvas using torque control while following a trajectory generated by the image processing method. This project is running on Gazebo Ignition physics simulation and using ROS2 as a middleware.

Keywords: Torque control, Dynamics model, Trajectory generation, Image processing, Edge detection, ROS2, Physic simulation

1 Overview

This project explore the development of a torque-controlled system for UR5e robot to draw in a simulation space. Rather than using just position conteol, this project focuses on simultaneously controlling both the motion of the robot and the contact force created by the pen onto the canvas. The system integrates trajectory generation with analytic inverse kinematics, differential kinematics and dynamics based torque control. All of the components will be implemented in Gazebo with ROS2 as the communication backbone.

This report consists of six main sections, excluding the Overview:

- **Software Architecture and Simulation**
  Describes the simulation environment, ROS2 ecosystem, and node–topic communication structure. Introduces the custom nodes (force–motion controller, IK node) and supporting nodes (state broadcaster, effort controller, ros–gz bridge, RViz visualization).
- **Trajectory Generation**
  Explains how to generate a trajectory for drawing by using image-processing techniques. This algorithm includes covers edge detection, path extraction, and conversion into a CSV file with positions, velocities, and acceleration of end-effector for Inverse Kinematic to solve.
- **Inverse Kinematic**
  Detailed into the IK method used for the UR5e via ur_analytic_ik library which using geometric IK decomposition which will be verify using forward kinematics and differential IK for joint velocities and accelerations is also will be compute via pinocchio library.
- **Dynamics and Control System**
  Presents the motion control system based on UR5 dynamics. Shows the PD controller for trajectory tracking and introduces the Z-axis PI force-elevation controller to manipulate the contact force. including the full control diagram.

- **Test and Evaluation**
  Discusses the experimental procedure for validating the drawing performance in simulation. By using the RMSE metric to evaluate both trajectory accuracy and force-control performance.

- **Conclusion**
  Summarizes the results, highlights the effectiveness of combining torque control with force regulation, and identifies future work.

2 Software architecture and Simulation

This work performed in Gazebo ignition physic simulation. The simulation simulates the UR5e, pen, gravity, collision etc. ROS2 middleware was used to interface both the simulation and the backbone of the robot system. The software architecture can be display as Node and Topic (as shown on Fig.1)

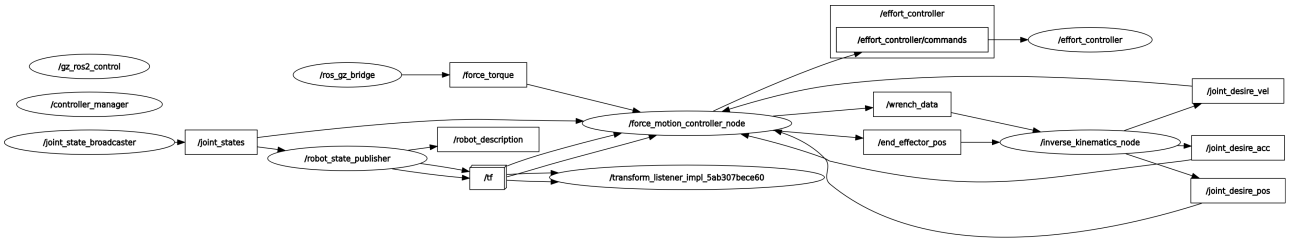

Figure 1: Nodes and topics

The `force_motion_controller_node` and `inverse_kinematic_node` were developed by the project team.

1. `force_motion_controller_node`
This node acquires and filters the wrench from the F/T sensor, reads end-effector positions, computes UR5e dynamics, and generates torques for motion control (can be found in `ur_controller` package).

2. `inverse_kinematic_node`
This node performs inverse kinematics calculations, Z-axis elevation force control and read trajectory from .csv file.

Additionally, There are several Node required to interface with the Gazebo

1. `robot_state_broadcaster`
This node publishes the current state (position, velocity) of individual joints to the `joint_states` topic

2. `robot_state_publisher`
This was used computes and broadcasts the 3D poses of a robotś links based on its URDF model and incoming `joint_states` messages

3. `ros_gz_bridge`
The F/T sensor is instant function from Gazebo ignition. To acquire wrench from F/T sensor, only need to bridge the `gz_topic` to the ROS2 topic using `ros_gz_bridge`

4. `effort_controller`
This node use to control the UR5e joint torque. This node is part of `ROS2_control` framework

Lastly, There is addition tools helps this project display drawing line and debugging

1. rviz2
This program show the 3D model of the UR5e, force and drawing line as show in Fig.2

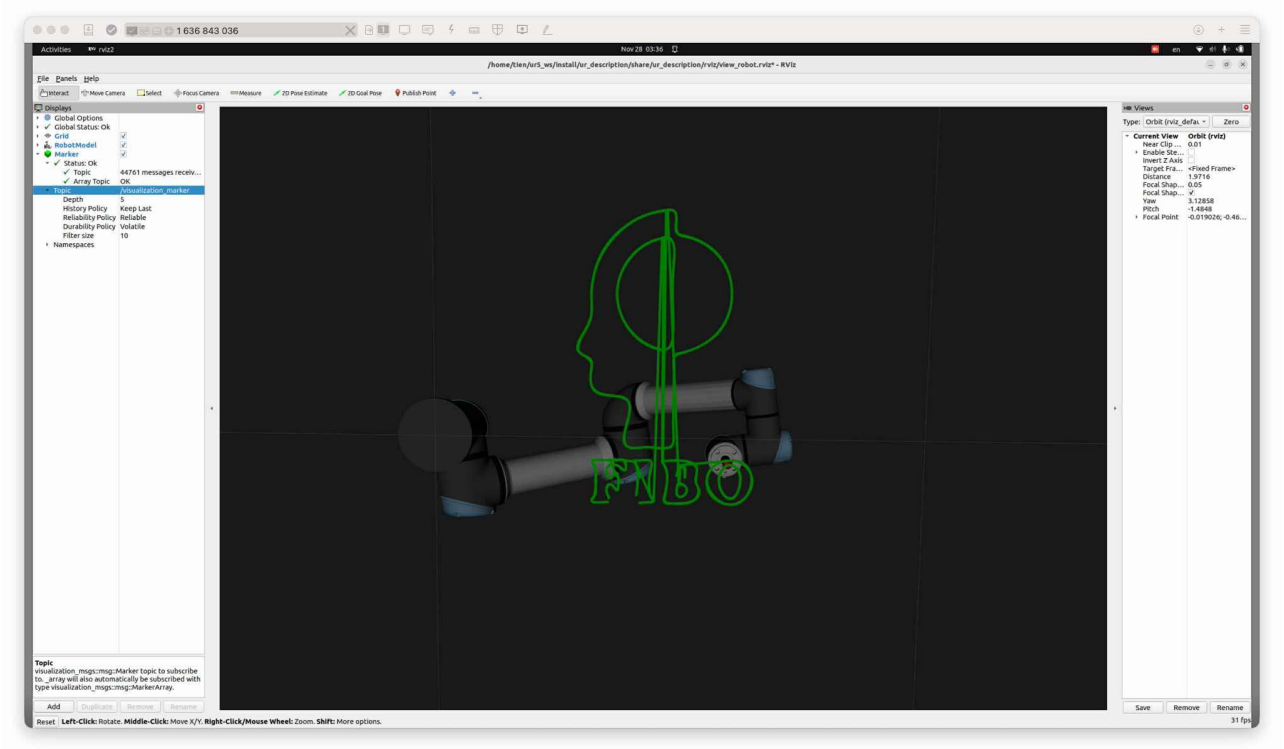

Figure 2: rviz2 interface

3 Trajectory generation

This section describes the transformation of a raw input image into a precise geometric path suitable for the robot's end-effector. Raw images contain significant noise and artifacts; if left unaddressed, these imperfections would propagate into the trajectory generation phase, causing the UR5e robot to execute erratic, jittery motions that increase mechanical wear and reduce drawing quality.

3.0.1 3.1 Noise Reduction

Raw input images often contain high-frequency noise and insignificant textures that are irrelevant to the drawing task. Before edge detection, we apply Gaussian Blur and Bilateral Filtering. This step is crucial for smoothing the input signal. By suppressing high-frequency noise while preserving structural edges, we prevent the generation of "phantom" waypoints that would cause the robot to vibrate or make unnecessary micro-movements during execution.

3.0.2 3.2 Canny Edge Detection

We utilize the Canny algorithm to identify the topological boundaries of objects within the image. This process converts the grayscale input into a binary map where edge pixels are represented as white (255) and the background as black (0). This establishes the fundamental geometry for the robot's path planning.

3.0.3 3.3 Addressing the "Double Line" Effect with Skeletonization

Standard edge detection often produces thick boundaries, resulting in the "Double Line Effect" where the algorithm interprets a single edge as two parallel paths. To resolve this, we apply Skeletonization, which collapses these structures into a single 1-pixel wide centerline. This eliminates redundant movements and ensures the robot traces a unique, clean path.

3.0.4 3.4 Path Planning and Sequence Generation

We utilize OpenCV's `findContours` function with `CHAIN_APPROX_NONE` to convert the binary pixel chains into a set of vector paths. This preserves every boundary point, ensuring the robot captures high-fidelity details of the original image without simplifying curves into straight lines. The default contour order follows a scan-line pattern (top-to-bottom), which causes inefficient cross-workspace travel. To minimize "air time," we implemented a Nearest Neighbor Greedy Algorithm, which uses logic to find the nearest point starting from the

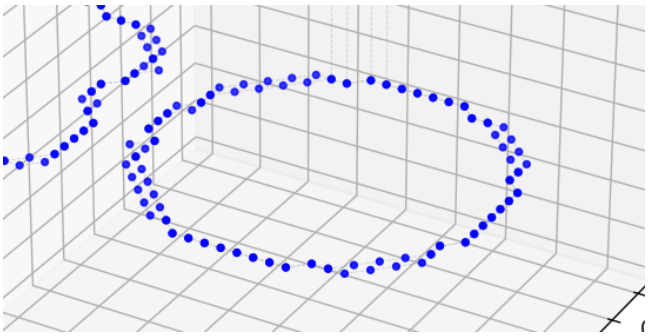

Figure 3: Double Line Effect

starting point. The algorithm iteratively selects unexplored contour lines whose starting point is closest to the robot's current end-effector position. This reordering creates a localized group of drawing paths, significantly reducing overall cycle time and unnecessary robot movements.

3.0.5 3.5 Time-Parameterized Trajectory Generation

Generating trajectories directly from raw contours can cause infinite acceleration at sharp corners. To address this, we impose operational limits rather than using the UR5e's theoretical maximums. To ensure precision and safety, we constrain **Drawing Velocity** ($V_{draw} \leq 0.05\,m/s$), **Travel Velocity** ($V_{travel} \leq 0.25\,m/s$), and limit acceleration (A_{max}) to prevent mechanical jerk.
To enforce these limits, we implemented an Iterative Time Scaling Algorithm. The system generates a trial trajectory and calculates peak velocity (v) and acceleration (a). If any value exceeds the safety limits, the segment duration (T) is scaled up proportionally ($a \propto 1/T^2$), and the trajectory is recomputed. This ensures the final motion is time-efficient while strictly adhering to physical constraints.

- **Hybrid Trajectory Strategy and Implementation**
  We developed a hybrid strategy using Python (`NumPy`, `SciPy`) to handle the distinct requirements of air travel versus surface drawing.

- **Discrete Motion for Travel (Quintic Polynomial)**
  For non-drawing movements (lifting, lowering, air travel), stability is paramount. We implemented a **Quintic Polynomial** generator. This 5th-order function guarantees zero velocity and acceleration at start/stop points ($v_0 = v_f = 0$), ensuring a "Minimum Jerk" profile.

$$p(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5 \tag{1}$$

  Additionally, vertical transitions (Lift/Lower) enforce a minimum duration (e.g., 0.5s) to guarantee surface clearance before lateral movement begins.

- **Continuous Motion for Drawing (Cubic Spline)**
  Stopping at every waypoint during drawing causes jitter. To achieve fluid strokes, we use **Cubic Spline Interpolation** with *clamped* boundary conditions. Unlike the point-to-point Quintic method, this fits a smooth curve through the entire coordinate sequence, maintaining continuous non-zero momentum throughout the stroke while ensuring a smooth start and stop at the line's endpoints.

$$p(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 \tag{2}$$

- **Trajectory Output**
  The culmination of this pipeline is a high-resolution CSV dataset containing the full kinematic state vector $(t, x, y, z, v_x, v_y, v_z, a_x, a_y, a_z)$ for every control cycle ($dt = 0.008s$). This pre-calculated trajectory serves as the direct input for the subsequent **Inverse Kinematics (IK)** solver. By providing not just position but also velocity and acceleration targets, the system enables the IK solver (or the robot controller) to compute precise joint-space commands $(\theta, \dot{\theta}, \ddot{\theta})$, ensuring the UR5e follows the generated path with high dynamic fidelity.

4 Inverse Kinematic

The Inverse Kinematics (IK) used to calculates the joint angles of the UR5e robotic arm to achieve the desired end-effector position and orientation.
This section describes the process for creating both IK and differential kinematics pathways that convert desired Cartesian positions (including velocity and acceleration) into a joint space. The IK implementation can be used in an ROS2 node that publishes a target for each of the robot's joints to that robot's downstream controller.

4.1 Inverse Kinematic Method

The UR5e is a 6-DoF serial manipulator with the joint layout:

1. Base rotation
2. Shoulder
3. Elbow
4. Wrist 1
5. Wrist 2
6. Wrist 3

This structure enables a closed-form inverse kinematics solution, which is implemented in the Python library ur_analytic_ik.
The ur_analytic_ik solver uses the UR5e's modified Denavit–Hartenberg (DH) parameters which are:

Joint	a_i (m)	d_i (m)	α_i (rad)
1	0.0	0.1625	$+\pi/2$
2	-0.425	0.0	0
3	-0.3922	0.0	0
4	0.0	0.1333	$+\pi/2$
5	0.0	0.0997	$-\pi/2$
6	0.0	0.0996	0

Table 1: DH parameters of the UR5e robot.

The UR analytic IK uses classical geometric IK decomposition which divided into 4 main steps:

1. **Solve Wrist Center Decomposition:** Compute the position of the wrist center by removing the effect of the tool extension along the end-effector z-axis:

$$\mathbf{WC} = \mathbf{P}_{EE} - d_6 \cdot \mathbf{R}_{EE}\hat{z}$$

   where $\mathbf{P}_{EE}$ and $\mathbf{R}_{EE}$ are the desired end-effector position and orientation, and d_6 is the wrist to tool distance.

2. **Solve Joint 1:** Determine the base rotation angle to align the arm with the wrist center:

$$\theta_1 = \arctan 2(WC_y, WC_x)$$

3. **Solve Joints 2 and 3:** Use planar geometry and the law of cosines to compute the shoulder and elbow joint angles. Let a_2 and a_3 be the link lengths:

$$\theta_2 = \arctan 2(z', r') - \arctan 2(a_3 \sin\theta_3, a_2 + a_3 \cos\theta_3), \quad \theta_3 = \arccos\left(\frac{r^2 - a_2^2 - a_3^2}{2a_2a_3}\right)$$

   where (r', z') is the projection of the wrist center onto the plane of joints 2 and 3.

4. **Solve Wrist Joints (4, 5, 6):** Compute the final three joint angles to achieve the desired end-effector orientation:

$$^3R_6 = (^0R_3)^\mathsf{T} \cdot {}^0R_6$$

   and the wrist angles $\theta_4, \theta_5, \theta_6$ are extracted from $^3R_6$ using standard Euler-angle decomposition.

And then each geometry of the manipulator and the properties of trigonometric functions give us 2 option, so the total solution for inverse kinematic is 2(elbow) $\times$ 2(shoulder) $\times$ 2(wrist) = 8 possible solutions.

Then we select the best solution by filtering only solution with elbow up pose then choose the solution with the configuration closest to the previous solution.

4.2 Inverse Kinematic verification

To verify inverse kinematic we used forwark kinematic that already coded in this library which is:

$$ {}^{0}T_{6} = {}^{0}T_{1}\,{}^{1}T_{2}\,{}^{2}T_{3}\,{}^{3}T_{4}\,{}^{4}T_{5}\,{}^{5}T_{6} $$

where each transform is:

$$ {}^{i-1}T_{i} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

and then compare it with the task space we used for inverse kinematic.

4.3 Differential Inverse Kinematics

To compute both joint velocities and acceleration, we need UR5e Jacobian matrix. And since ur_analytic_ik doesn't have Jacobiancomputation included. Pinocchio library is used for Jacobian computations.

4.3.1 Joint Velocities

The joint velocities are calculated from the desired end-effector velocity using the UR5e Jacobian matrix for a given joint configuration.
End-effector velocity $\mathbf{V}$ is related to the joint velocities $\dot{\mathbf{q}}$ as:

$$ \mathbf{V} = J(\mathbf{q})\,\dot{\mathbf{q}} $$

Rearranging gives the joint velocities:

$$ \dot{\mathbf{q}} = J^{\dagger}\mathbf{V} $$

4.3.2 Joint Accelerations

The joint accelerations are calculated from the desired end-effector acceleration using the change in the Jacobian over time:

$$ \mathbf{A} = J(\mathbf{q})\,\ddot{\mathbf{q}} + \dot{J}(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}} $$

Rearranging gives the joint accelerations:

$$ \ddot{\mathbf{q}} = J^{\dagger}\left(\mathbf{A} - \dot{J}\,\dot{\mathbf{q}}\right) $$

5 Dynamics and Control system

There are various type of control method available for UR5e with torque control. This work presents a motion control strategy based on the dynamics of the UR5e. To generate the required torque for motion control, The inverse dynamics equation in joint space (equation 1) is used to compute required torque respect to the Joint state[1]. To control position, velocity and acceleration of the UR5e using PD motion controller (equation 2)

$$\tau = M(q)\ddot{q} + C(q, \dot{q}) + g(q) \tag{3}$$

$$\ddot{q} = \ddot{q_d} + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) \tag{4}$$

where:

- τ is the joint torque Vector
- $M(q)\ddot{q}$ is the Inertia Matrix
- $C(q, \dot{q})$ is the Coriolis and Centrifugal Matrix
- $g(q)$ is the Gravity Vector
- $\ddot{q}$ is the computed acceleration (command)
- K_p and K_d are the proportional and derivative gain matrices
- q_d and q are the desired and actual joint positions
- $\dot{q}_d$ and $\dot{q}$ are the desired and actual joint velocities
- $\ddot{q_d}$ is the desired joint acceleration

However, By using the dynamics model in joint space (equation 1) and PD controller (equation 2) are not able to control the force occurs at the end-effector. The method for control motion and force at end-effector simultaneously is to apply the Hybrid force-motion control. Even so the hybrid force-motion control is using the dynamics model in task space. Make the dynamics calculation more complicated to compute and using dynamics model in joint space for motion controller and force controller together will make controller fighting over position. This project presents the Z-axis elevation PI control (equation 3) method to control the force occurs at the end-effector. F/T sensor is noisy even after apply the low-pass filter. This cause the force elevation to use only PI controller because the D term is noise sensitive. This method will not affect the control system to compensate the position error with stronger force respect to the time because of the motion controller is only PD controller.

$$Z = -1 \cdot (K_p(W_d - W) + K_i \int (W_d - W)\frac{d}{dt}) \tag{5}$$

where:

- Z is the elevation in Z axis (task space)
- K_p and K_i are the proportional and integral gain matrices
- W_d and W are the desired and actual force at end-effector Z axis

After apply PD motion controller and PI elevation controller, UR5e are able to control motion and force simultaneously without fighting between 2 controllers in joint space. The control diagram is show in Fig.4. Lastly the inverse dynamics model calculation done by Pinocchio framework on python. And Controlled by the custom code written by this project team.

[1] Joint state: joint position, joint velocity, joint acceleration

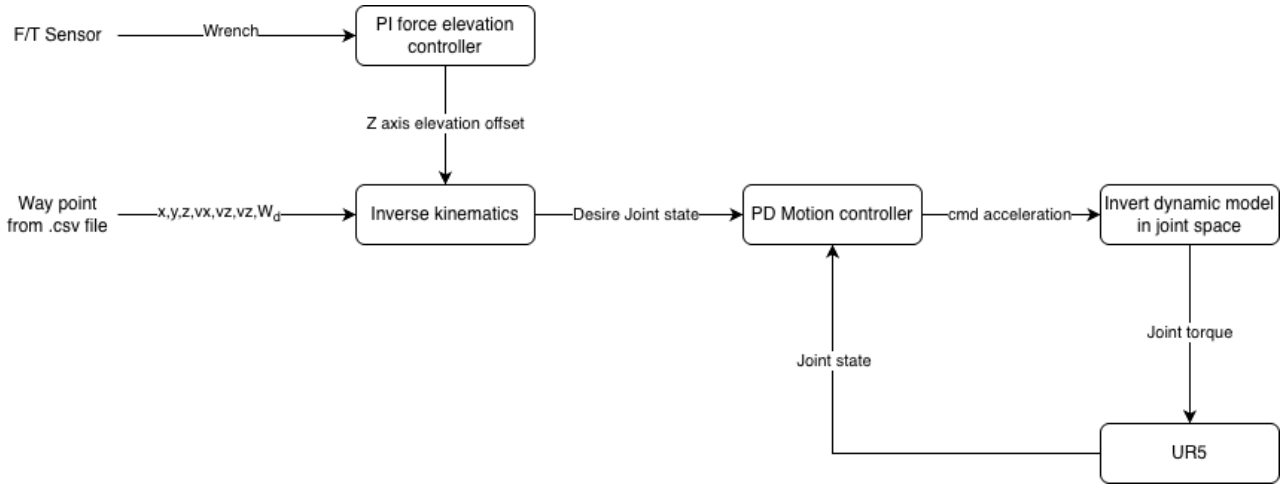

Figure 4: Control system diagram

6 Test and Evaluation

The requirement of this project is to perform drawing task within position error 10% range and able to do force control. RMSE (Root Mean Square Error) statistic method can be used to determine the average position error occurs on the system when UR5 done drawing task. With RMSE calculation code, Computer can average the RMSE of each joint over time while UR5 is operated drawing task and when UR5 is done drawing task. **The average position RMSE of this control system are 0.0022 radian or 1.9% error from joint limit**. The result of the controller can be seen as drawing in Fig.5

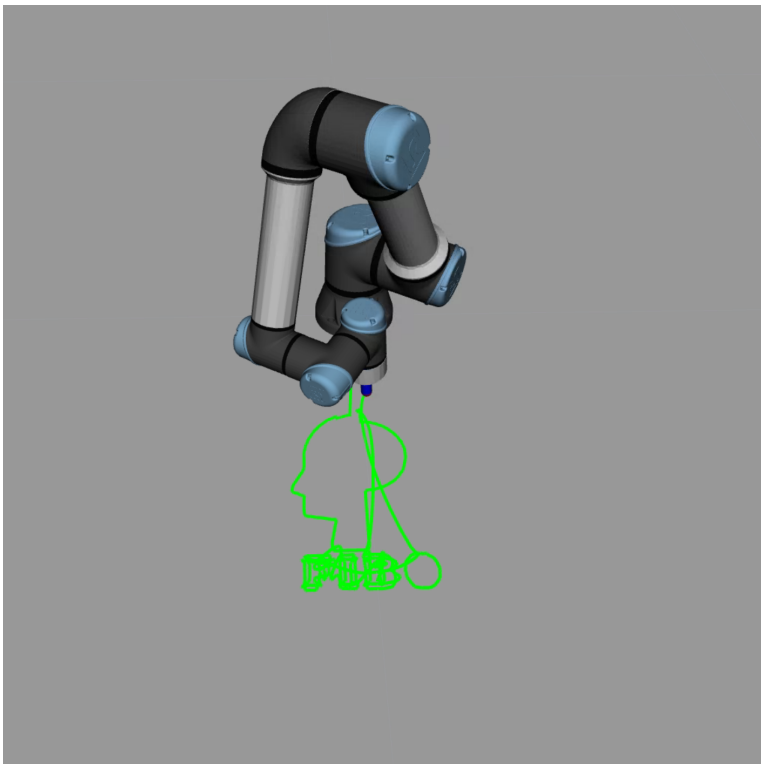

Figure 5: drawing

The force at the end effector can be achieving desire force and can maintain the force while UR5e perform drawing task as show in Fig.6. After several tests the force elevation controller unable to perform more that desire z-axis force at -1 N (while end-effector is not touching floor, The force at end-effector is around -3.5 N). After crossing the -1 N boundary, The UR5e can't stabilize the end-effector orientation anymore as show in Fig.7.

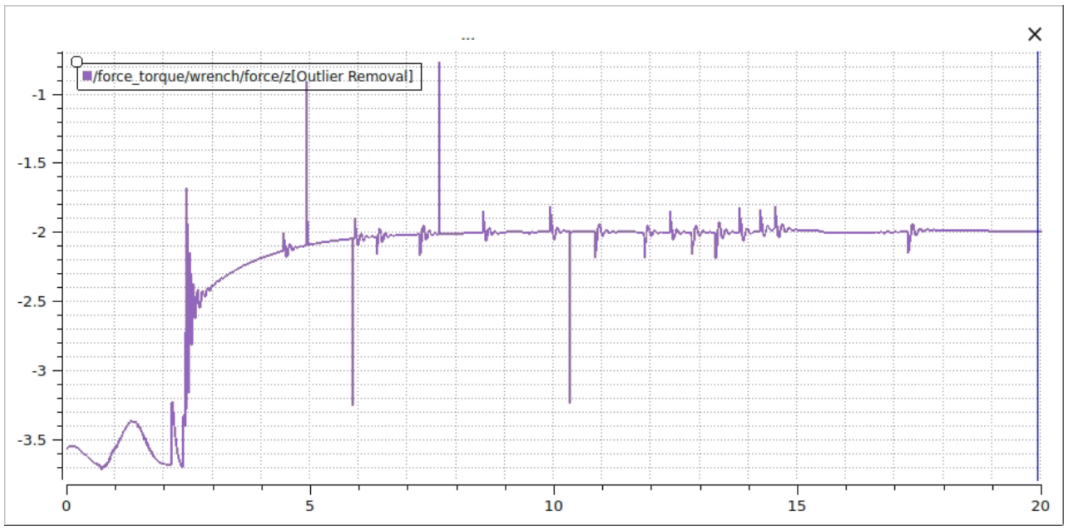

Figure 6: z-axis end-effector force at desire -2 N while UR5e perform drawing task

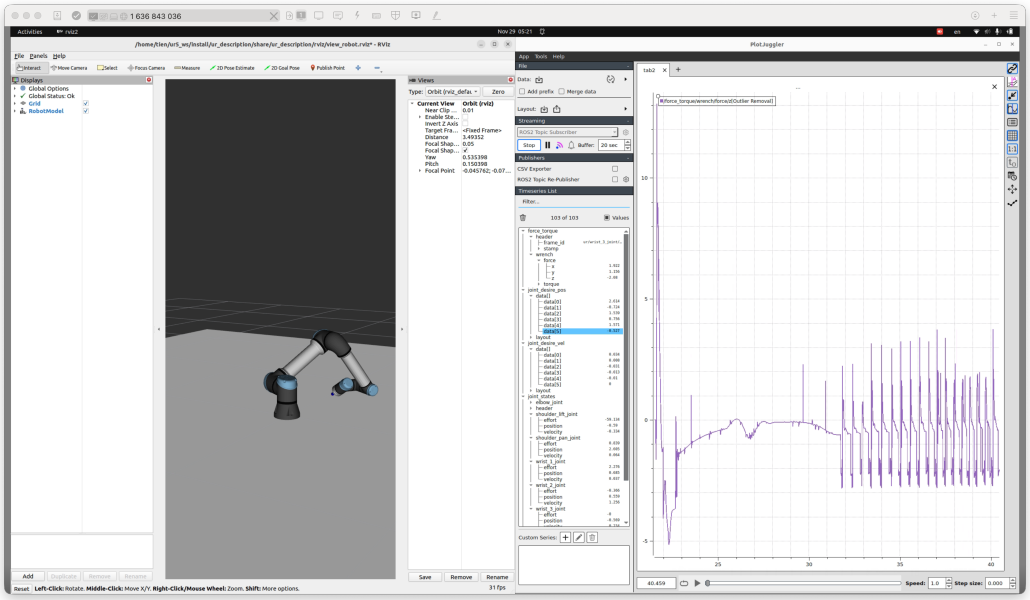

Figure 7: z-axis end-effector force at desire 0 N while UR5e perform drawing task

7 Conclusion

This project successfully implemented a control system incorporating a motion controller and a force controller simultaneously for the UR5e robot in the Gazebo Ignition physics simulation. The primary objective was to perform a drawing task while maintaining a position tracking error of less than 10% and controlling the contact force applied to the canvas. The system achieved high accuracy, with the motion control system recording an average Root Mean Square Error (RMSE) of 0.0022 radians, which is only 1.9% error from the joint limit. The trajectory generation process, starting from any image, effectively created the required waypoints. Furthermore, the Inverse Kinematics successfully converted the task space targets (position, velocity, and acceleration) into joint space commands. Regarding force regulation, the force control mechanism was able to achieve the desired force but was constrained to a small effective range of 0–2 N. A key limitation found was that if the desired force extended beyond these boundaries, the robot was unable to maintain the necessary end-effector orientation

8 Material

1. Demo Video YouTube link
2. GitHub